UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA – UTEC

CS4052 – Computación Paralela y Distribuida – 2026-II

Paralelización del clasificador KNN con MPI

Proyecto 5: `load_digits`, PRAM, escalabilidad y límites del hardware



Integrantes:
- Juan David Velo Poma
- Farid Jack Aquino Castro
- Josue Nehemias Velo Poma

Profesor: José Fiestas, PhD

Lima, Perú

2026

Ruta de la presentación

1. Problema y diseño paralelo
KNN, metodología PCAM, PRAM CREW y algoritmos con `pardo`.

2. Implementación y experimentos
Tres versiones beta, variante DAG, entorno y protocolo de medición.

3. Resultados y análisis
Tiempos, speedup (Amdahl), normalización y s^*, FLOP/s, escalabilidad débil (Gustafson).

4. ¿Por qué se satura?
Comunicación lineal vs. árbol, núcleos P/E, SMT y conclusiones.

Pregunta central

¿Cuánto se puede acelerar KNN repartiendo las consultas entre procesos MPI en un procesador multinúcleo, y **qué limita** esa aceleración?

Respuesta corta: hasta $\sim 5.3\times$, con un óptimo en $s^* \approx 12$ procesos; el límite es el **bus de memoria**, no MPI.

PARTE 1

Problema y diseño paralelo

KNN, PCAM, PRAM CREW y formalismo `pardo`

1

El problema: KNN y por qué paralelizarlo

K vecinos más cercanos (KNN)

- Clasificador **no paramétrico**: no entrena; todo el costo está en la consulta.
- Para cada consulta q: calcular n_{train} distancias, tomar las $k = 3$ menores y votar.
- `load_digits`: 1797 imágenes 8×8, $d = 64$, 10 clases; $n_{\text{train}} = 1437$, $m_{\text{test}} = 360$.

Distancia euclidiana (región paralelizable)

$$d(q, x) = \sqrt{\sum_{l=1}^{d} (q_l - x_l)^2} \quad \Rightarrow \quad 3d \text{ FLOPs por par}$$

Trabajo total: $W = 3\, m_{\text{test}}\, n_{\text{train}}\, d$.

¿Por qué es ideal para paralelizar?

- Cada consulta es **independiente** de las demás.
- En el DAG: distribución $\rightarrow$ fase masivamente paralela $\rightarrow$ recolección.
- Trabajo concentrado en una sola región bien identificada.

Desafío metodológico

El secuencial tarda 1.4 s, pero vectorizado tarda **milisegundos**: el arranque de MPI dominaría. **Solución:** datos sintéticos con la estadística de cada clase (N hasta 245 000).

Diseño con la metodología de Foster (PCAM)

1. Particionamiento

Tarea primitiva: una distancia $d(q, x) \Rightarrow m_{\text{test}} \cdot n_{\text{train}}$ tareas.

2. Comunicación

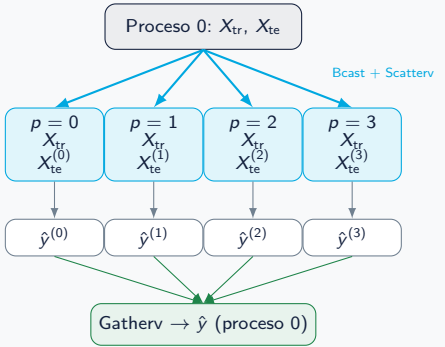
X_{tr} completo a todos (`Bcast`); cada consulta a un proceso (`Scatterv`); etiquetas a la raíz (`Gatherv`).

3. Aglomeración

$m_p \approx m_{\text{test}}/s$ consultas por proceso $\Rightarrow$ granularidad $G = T_{\text{comp}}/T_{\text{comm}}$ alta.

4. Mapeo

Un proceso MPI por núcleo; bloques con m_{test} mód $s \neq 0$.



Taxonomía de Flynn: MIMD con paradigma SPMD (mismo programa, datos distintos).

Modelo PRAM CREW y teorema de Brent

Algoritmo 4: KNN en PRAM CREW

```
ŷ ← 0 ∈ {0, ..., 9}^{m_test};
forall j ← 1 to m_test pardo
    ŷ[j] ← knn_predict(X_te[j], X_tr, y_tr, k);
exactitud ← mean(ŷ = y_te);
```

- **CR** (lectura concurrente): todos leen X_{tr} e y_{tr}.
- **EW** (escritura exclusiva): el procesador j escribe sólo $\hat{y}[j]$.

Trabajo y span

$$W = T_s = O\big(m_{\text{test}}(n_{\text{train}}\, d + n_{\text{train}} \log n_{\text{train}})\big)$$

$$T_\infty = O(n_{\text{train}}\, d + n_{\text{train}} \log n_{\text{train}})$$

Teorema de Brent (s procesadores)

$$T_{\text{comp}}(s) \leq \frac{W}{s} + T_\infty$$

Con $s \leq m_{\text{test}}$ domina W/s: el cómputo escala idealmente en el modelo. La diferencia con la realidad la explicaremos en la Parte 4.

Algoritmo distribuido con formalismo `pardo`

Algoritmo 5: KNN distribuido con MPI (Beta 3)

forall $p \leftarrow 0$ **to** $s-1$ **pardo**
  ; ▷ $\log s\,(\alpha + n_{\texttt{train}} d\,\beta)$
  $X_{\text{tr}}, y_{\text{tr}} \leftarrow \texttt{Bcast}(X_{tr}, y_{tr}, \textit{raíz} = 0)$;
  ; ▷ $\alpha \log s + m_{\texttt{test}} d\,\beta$
  $X_{\text{te}}^{(p)} \leftarrow \texttt{Scatterv}(X_{te}, \textit{counts}, \textit{displs})$;
  forall $j \leftarrow 1$ **to** m_p **pardo**
    $\hat{y}^{(p)}[j] \leftarrow \texttt{knn_predict}(X_{te}^{(p)}[j], X_{tr}, y_{tr}, k)$;
  ; ▷ `EW en bloques disjuntos`
  $\hat{y}[\text{displ}_p..] \leftarrow \texttt{Gatherv}(\hat{y}^{(p)}, \textit{counts}, \textit{displs})$;

Dos niveles de paralelismo

- **forall** p: los s procesos MPI (SPMD).
- **forall** j: consultas locales independientes, ejecutadas como producto matricial (BLAS).

Reparto exacto

$m_p = \lfloor m_{\text{test}}/s \rfloor + [\, p < m_{\text{test}} \bmod s \,]$

Dos descomposiciones: replicar $X_{\mathbf{tr}}$ vs. variante del DAG

Beta 3: replica $X_{\mathbf{tr}}$, reparte consultas

`Bcast`(X_{tr}) $\rightarrow$ `Scatterv`(X_{te}) $\rightarrow$ predicción $\rightarrow$ `Gatherv`($\hat{y}$)

$$T_{\text{comm}} = O\big(\alpha \log s + \beta(n_{\text{train}}\, d \log s + m_{\text{test}}\, d)\big)$$

Cada proceso obtiene el vecino global **sin fase de fusión**.

Variante DAG: reparte $X_{\mathbf{tr}}$, difunde consultas

`Scatterv`(X_{tr}) $\rightarrow$ `Bcast`(X_{te}) $\rightarrow$ k candidatos locales $\rightarrow$ `Gather` $\rightarrow$ fusión

$$T_{\text{comm}} = O\big(\alpha \log s + \beta(n_{\text{train}}\, d + m_{\text{test}}\, d \log s + m_{\text{test}}\, k s)\big)$$

Más una **fusión serial** $O(m_{\text{test}}\, s\, k)$ en la raíz.

Corrección de la variante DAG. Si x está entre los k vecinos globales de q y pertenece al bloque p, también está entre los k vecinos de q *dentro* de ese bloque. Por tanto, la unión de los $s\,k$ candidatos contiene a los k globales: **misma predicción que el secuencial**.

PARTE 2

Implementación y experimentos

Tres versiones beta, validación y protocolo de medición

Desarrollo en tres versiones beta

Versión	Cambio clave	T ($s = 8$)	Speedup
Secuencial	bucle Python (enunciado)	1.401 s	1.0
Beta 1	`bcast/scatter/gather` pickle	0.210 s	6.7
Beta 2	NumPy vectorizado + $T_{\text{comp}}/T_{\text{comm}}$	0.0030 s	467
Beta 3	`Bcast/Scatterv/Gatherv` buffers	0.0020 s	700
DAG	reparte X_{tr} + fusión	0.0014 s	1014

Tiempos sobre `load_digits` con 8 procesos; speedup respecto del secuencial original.

Validación

- Exactitud **0.9833** en todas las versiones (= secuencial).
- Beta 3 y DAG: **100 %** de predicciones iguales a `KNeighborsClassifier` (scikit-learn).
- Probado con m_{test} mód $s \neq 0$ (360 consultas entre 7 procesos).

Lección

Vectorizar (Beta 2) gana $\sim$156$\times$; los buffers contiguos (Beta 3) un 33 % más. La granularidad cae y por eso se escala N.

Entorno experimental y protocolo

Plataforma

- Intel Core i7-14650HX: **8 P-cores** (con SMT) + **8 E-cores** = 24 hilos.
- 32 GB DDR5-5600 (un solo nodo de memoria).
- Python 3.11, mpi4py 4.1.2 sobre Intel MPI; BLAS con 1 hilo.

Datos sintéticos

Por clase: $x \mid c \sim \mathcal{N}(\mu_c, \Sigma_c)$ estimadas de `load_digits`; $d = 64$, 10 clases, valores en $[0, 16]$.

Barridos

- $s \in \{1, 2, 4, 8, 12, 16, 20, 24\}$.
- **Fuerte:** $N = 50\text{k}, 100\text{k}, 200\text{k}$.
- **Débil:** $N = N_0\sqrt{s}$ y $N = N_0\, s$.
- 291 ejecuciones; mediana de 3–7 repeticiones.

Experimentos complementarios

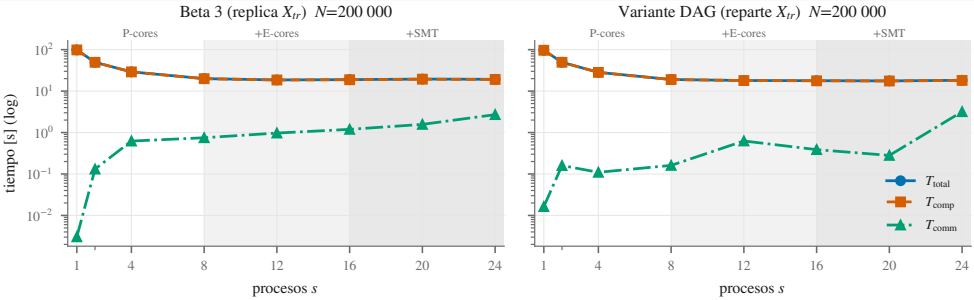
- Colectivas **sin cómputo** $\Rightarrow \alpha$, β.
- Afinidad a P-cores, E-cores o hilos SMT con carga fija.

PARTE 3

Resultados y análisis

Tiempos, speedup, normalización, FLOP/s y escalabilidad

Tiempos de ejecución, cómputo y comunicación



N = 200 000 (n_{train} = 160 000, m_{test} = 40 000). Bandas: P-cores ($s \leq 8$), +E-cores, +SMT ($s > 16$).

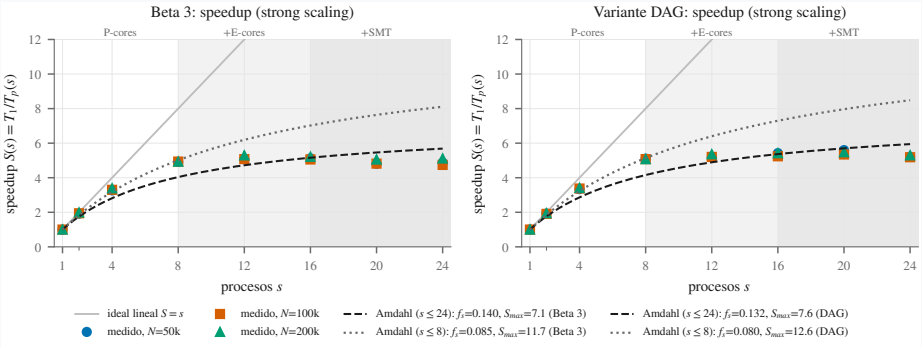
98.5 s → 18.6 s
T_{total} con $s = 1$ y $s = 12$

>99 %
del tiempo es cómputo

Observación clave

Desde $s = 8$, T_{comp} **deja de bajar**: cada proceso hace W/s operaciones pero tarda lo mismo. El crecimiento de T_{comm} para $s \geq 16$ es espera por desbalance.

Speedup y Ley de Amdahl (escalabilidad fuerte)



5.29
S máximo medido ($s = 12$)

Ley de Amdahl

$$S(s) = \frac{1}{f_s + f_p/s}$$

$$S_{\text{máx}} = 1/f_s$$

Ajuste	f_s	$S_{\text{máx}}$
$s \le 24$	0.140	7.1
$s \le 8$	0.085	11.7

$f_s \approx 0.14$ **no es código serial** (éste es menor al 1 %): es una fracción serial *efectiva* por **contención del bus DDR5**. Los tres N dan la misma curva.

Normalización del modelo teórico y proceso óptimo s^*

Modelo normalizado

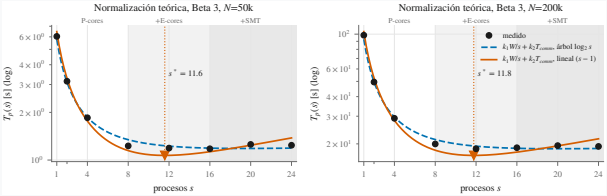
$$T_p(s) = k_1 \frac{W}{s} + k_2\, T_{\text{comm}}(s)$$

$W = 3\, m_{\text{test}}\, n_{\text{train}}\, d$. Lineal: $T_{\text{comm}} = (s-1)(\alpha + M\beta)$;
árbol: $T_{\text{comm}} = \log_2 s\,(\alpha + M\beta)$.

Procedimiento

1. α, β medidos con colectivas aisladas.
2. k_1, k_2 por mínimos cuadrados sobre T medido.
3. $\dfrac{dT_p}{ds} = -\dfrac{k_1 W}{s^2} + k_2 c = 0$

$$\Rightarrow \boxed{s^* = \sqrt{\frac{k_1 W}{k_2\, c}}}$$



Modelo (N=200k)	k_1 [s/FLOP]	k_2	error	s^*
Árbol	7.6×10^{-11}	139	3.4 %	19.9
Lineal	8.4×10^{-11}	122	6.4 %	**11.8**
Mínimo experimental				**12**

$s^*_{\text{teórico}} = 11.8$ **vs.** $s^*_{\text{medido}} = 12$
modelo de comunicación lineal, $N = 200$k

Interpretación de la normalización

$1/k_1 \approx 12$ **GFLOP/s**

Es el rendimiento medido de **un P-core aislado**. Estable entre los tres tamaños de problema.

$k_2 \gg 1$ **y crece con** N

Si sólo costaran los mensajes, $k_2 \approx 1$. El exceso es la **competencia por el bus de memoria**: cada proceso recorre X_{tr} (80 MB $>$ 30 MB de L3). Cuello de botella de **von Neumann**.

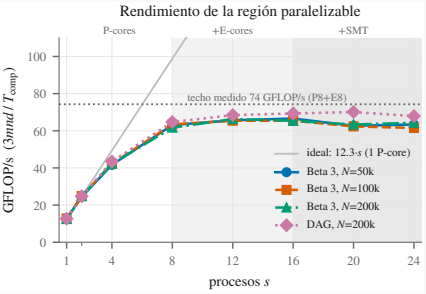
s^* **no depende de** N

$W \propto N^2$ y $k_2 c \propto N \cdot N \Rightarrow$ $s^* \approx 11.4$–11.8 para 50k, 100k y 200k, igual al mínimo medido ($s = 12$).

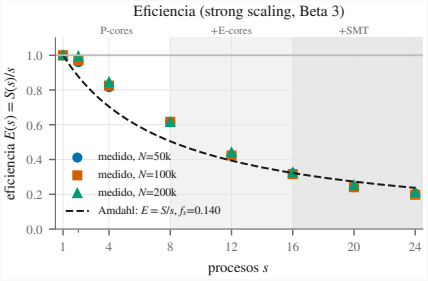
Cantidad óptima de procesos: $s^* \approx 12$ (8 P-cores + 4 E-cores)

Entre $s = 12$ y $s = 24$ el tiempo varía menos del 5 %: mismo resultado con la mitad de los recursos.

Rendimiento en FLOP/s y eficiencia



GFLOP/s $= 3\, m_{\text{test}} n_{\text{train}} d \;/\; (T_{\text{comp}} \cdot 10^9)$.



$E(s) = S(s)/s$ medida y predicha por Amdahl.

12.5
GFLOP/s con $s = 1$

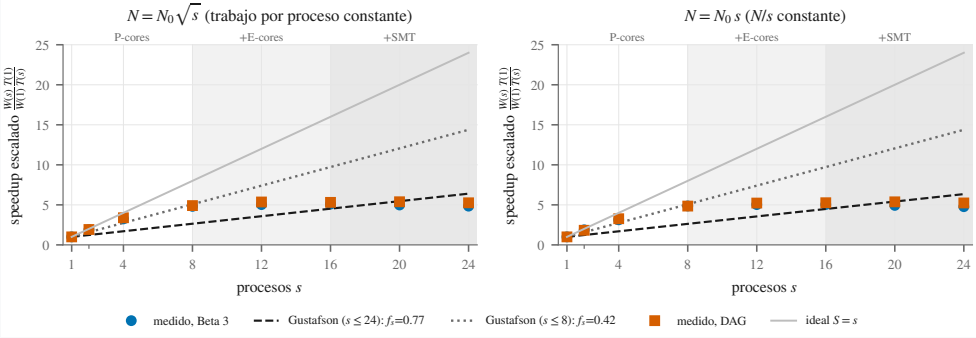
66 / 70
GFLOP/s Beta 3 / DAG

74.3
GFLOP/s techo medido

0.21
eficiencia $E(24)$

Con n_{train} fijo $E(s) \to 0$: el algoritmo **no es fuertemente escalable** en esta máquina. El rendimiento depende de los núcleos activos y de su acceso compartido a memoria, **no de** N.

Escalabilidad débil y Ley de Gustafson



Ley de Gustafson

$S(s) = f_s + s\, f_p$

0.20

$E_{\text{débil}}(24)$

Detalle clave: $W \propto N^2$

N/s constante **no** fija el trabajo por proceso ($W/s \propto s$). Lo correcto es $N = N_0\sqrt{s}$.

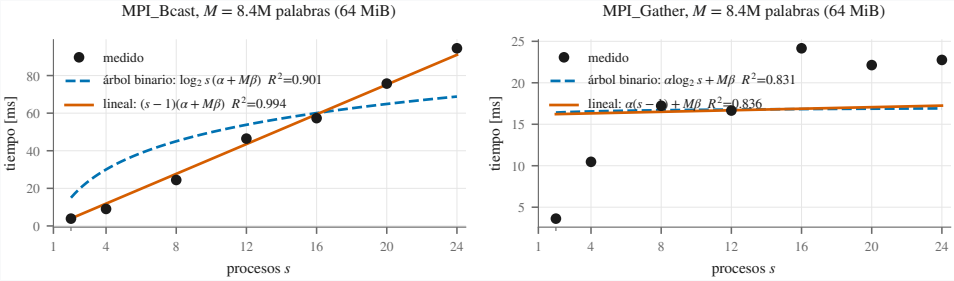
Con trabajo constante: 5.9 → 29.4 s ($E = 0.20$ con $s = 24$), $f_s = 0.42$ ($s \le 8$). Aumentar N **no añade canales de memoria**.

PARTE 4

¿Por qué se satura?

Comunicación lineal vs. árbol, núcleos P/E y SMT

Modelo de comunicación: lineal vs. árbol binario



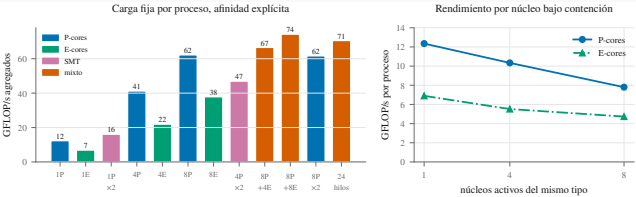
Colectivas sin cómputo, mensaje de 64 MiB. Ajuste con 9 tamaños de mensaje y 7 valores de s.

`Bcast`	Modelo (Unidad 2)	R^2
Árbol	$O(\log p\,(\alpha + m\beta))$	0.901
Lineal	$O(p\,(\alpha + m\beta))$	**0.994**

De $s = 2$ a $s = 24$ el `Bcast` crece **24**× (3.9 → 94.5 ms): es $O(s)$, no $O(\log s)$.

Sin enlaces independientes, las $s-1$ copias **compiten por el bus DDR5**. En el KNN el `Bcast` es menos del 1 % del tiempo.

Justificación de hardware: núcleos P, núcleos E y SMT



Misma carga por proceso, cada proceso fijado a una CPU lógica concreta.

> *"No se recomienda usar todos los hilos de un CPU [...] Pasado cierto punto, cada hilo extra aporta poco o incluso resta. Debido a: Ancho de banda de memoria, [...] SMT no equivale a núcleos reales, [...] Desbalance de carga. El óptimo suele estar en un punto intermedio."* — Fiestas, Unidad 4, pág. 22

1. Ancho de banda DRAM saturado

P-core: 12.3 GFLOP/s solo → 10.3 (4 activos) → 7.8 (8 activos).

2. SMT no es núcleo real (+30 %)

2 hilos en 1 P-core: +30 %. 8 P físicos: 62 vs. 4P×2: 47 GFLOP/s.

3. Desbalance P-core vs. E-core

E-core = mitad de un P-core (6.9 vs. 12.3). Los rápidos esperan en `Gatherv`.

$s^* = 12$ es el "punto intermedio" de la diapositiva del curso.

Beta 3 vs. variante DAG y mejoras propuestas

s	Beta 3 [s]	DAG [s]	Fusión DAG [s]
1	98.46	96.52	0.006
8	19.97	19.03	0.018
12	18.60	18.01	0.025
16	18.90	17.82	0.043
24	19.26	18.23	0.056

$N = 200\,000$. Ambas producen predicciones idénticas.

DAG: 2–6 % más rápida

Evita el `Bcast` de X_{tr} y cada proceso recorre sólo n_{train}/s puntos $\Rightarrow$ **menos tráfico a memoria**. La fusión serial es despreciable con $k = 3$.

Mejoras propuestas

- **Reparto ponderado** en `Scatterv`: P-core $\approx 2\times$ E-core.
- **Afinidad explícita** a P-cores físicos con $s \leq 8$.
- **Bloqueo por caché**: reutilizar bloques de X_{tr} en L2 para varias consultas.
- **Memoria distribuida** (clúster): cada nodo aporta su canal de memoria; ahí aplica el modelo en árbol y Gustafson.

Conclusiones

1. Diseño correcto

PCAM + PRAM CREW con `forall ... pardo`. Tres betas y DAG validadas al 100 % contra el secuencial y scikit-learn.

2. Desarrollo incremental

Vectorización $\sim$156×, buffers +33 %; hasta $\sim$1000× frente al código original con 8 procesos.

3. Teoría vs. experimento

$T_p = k_1 W/s + k_2 T_{\text{comm}}$ (modelo lineal): error 6 % y $s^* = 11.8$ vs. 12 medido.

4. Límite de escalabilidad

$S_{\text{máx}} \approx 5.3$ ($f_s = 0.14$), $E(24) = 0.21$; Gustafson tampoco se cumple. La causa demostrada: bus de memoria, SMT y desbalance P/E, **no MPI**.

5. Comunicación en memoria compartida

Las colectivas siguen el modelo **lineal** $O(s)$ ($R^2 = 0.994$): cuello de botella de von Neumann.

Recomendación

Usar $s \approx 12$ procesos, no los 24 hilos disponibles.

Fuentes y Bibliografía Destacada

Base teórica

J. Fiestas, *Unidades 1–4: Modelos PRAM, MPI, Leyes de Escalabilidad y Arquitecturas Híbridas*, CS4052, UTEC, 2026.

Desarrollo MPI

L. Dalcín, *MPI for Python (mpi4py) documentation*, 2025.

Leyes de escalabilidad

G. M. Amdahl (1967), *Validity of the Single Processor Approach...*

J. L. Gustafson (1988), *Reevaluating Amdahl's Law*.

Especificaciones de hardware

Intel Corp., *Intel Core i7-14650HX Processor Specifications*, 2024.

Bibliografía completa (36 referencias) en la Sec. 5 del informe final.

¡Muchas gracias por su atención!

Paralelización del clasificador KNN con MPI

Juan David Velo Poma · Farid Jack Aquino Castro · Josue Nehemias Velo Poma

Universidad de Ingeniería y Tecnología (UTEC) – 2026

PARTE A

Anexo

Diapositivas de respaldo para la ronda de preguntas

Anexo: Algoritmo Beta 1 (Comunicación básica)

```python
# Rango 0: carga, divide y prepara bloques (m % s != 0 admitido)
if rank == 0:
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42)
    test_chunks = np.array_split(X_test, size)

X_train = comm.bcast(X_train, root=0) # pickle: replica X_tr
y_train = comm.bcast(y_train, root=0)
X_local = comm.scatter(test_chunks, root=0) # bloque de consultas

# Cómputo local IDÉNTICO al secuencial: bucle Python por punto
def knn_predict(test_point, X_train, y_train, k):
    distances = [euclidean_distance(test_point, x) for x in X_train]
    k_indices = np.argsort(distances)[:k]
    k_labels = [y_train[i] for i in k_indices]
    return Counter(k_labels).most_common(1)[0][0]

y_local = [knn_predict(x, X_train, y_train, k) for x in X_local]
gathered = comm.gather(y_local, root=0) # pickle: recolección
```

Idea

Paralelizar **sin tocar el cómputo**: sólo cambia la descomposición de datos.

Colectivas genéricas

`bcast`, `scatter`, `gather` en minúscula serializan objetos con *pickle*.

Resultado

Exactitud 0.9833; $S(8) = 6.7$ ($E = 0.83$) sobre el secuencial.

Anexo: Algoritmo Beta 2 (Vectorización local)

```python
def knn_predict_block(X_train, y_train, sq_train, Q, k):
    # ||q - x||^2 = ||q||^2 + ||x||^2 - 2 q.x (b x n, sin bucles)
    D2 = np.einsum("ij,ij->i", Q, Q)[:, None] + sq_train[None, :] \
         - 2.0 * (Q @ X_train.T) # producto matricial (BLAS)
    D = np.sqrt(np.maximum(D2, 0.0)) # distancia euclidiana
    nn = np.argpartition(D, k - 1, axis=1)[:, :k] # k vecinos en O(n)
    votes = np.zeros((Q.shape[0], 10), dtype=np.int64)
    np.add.at(votes, (np.arange(Q.shape[0])[:, None], y_train[nn]), 1)
    return votes.argmax(axis=1) # moda vectorizada

t0 = MPI.Wtime()
X_train = comm.bcast(X_train, root=0) # comunicación aún pickle
t1 = MPI.Wtime()
X_local = comm.scatter(test_chunks, root=0)
t2 = MPI.Wtime()
y_local = knn_local(X_train, y_train, X_local, k, block=256)
t3 = MPI.Wtime() # T_comp = t3 - t2
gathered = comm.gather(y_local, root=0)
```

Cambio clave

El bucle Python de n_{train} iteraciones se reemplaza por `np.einsum` y un producto matricial por bloques de 256 consultas.

Instrumentación

`MPI.Wtime()` por fase: T_{comp} y T_{comm} separados (máximo entre procesos).

Resultado

$\sim$156$\times$ más rápida que la Beta 1 con $s = 1$; misma exactitud (0.9833).

Respaldo: derivación de s^* en ambos modelos

Modelo lineal

$$T_p(s) = k_1 \frac{W}{s} + k_2\, c\,(s-1), \quad c = \alpha + M\beta$$

$$\frac{dT_p}{ds} = -\frac{k_1 W}{s^2} + k_2 c = 0 \;\Rightarrow\; s^* = \sqrt{\frac{k_1 W}{k_2 c}}$$

N=200k: $k_1 W = 103.8\,\text{s}$, $k_2 c = 0.748\,\text{s} \Rightarrow s^* = 11.8$.

Modelo en árbol

$$T_p(s) = k_1 \frac{W}{s} + k_2\, c \log_2 s$$

$$-\frac{k_1 W}{s^2} + \frac{k_2 c}{s \ln 2} = 0 \;\Rightarrow\; s^* = \frac{k_1 W \ln 2}{k_2 c}$$

Predice $s^* \approx 20$ (zona SMT): no reproduce el aumento de T_p tras el mínimo.

$M = n_{\text{train}}(d+1) + m_{\text{test}} d + m_{\text{test}}$ palabras (`Bcast` de $X_{\text{tr}}, y_{\text{tr}}$ + `Scatterv` de X_{te} + `Gatherv` de $\hat{y}$). $W = 3\, m_{\text{test}} n_{\text{train}} d = 1.23 \times 10^{12}$ FLOPs con $N = 200$k.

Respaldo: escalabilidad fuerte completa ($N = 200\,000$, Beta 3)

s	T_{total} [s]	T_{comp} [s]	T_{comm} [s]	$S(s)$	$E(s)$	GFLOP/s
1	98.46	98.46	0.003	1.00	1.00	12.5
2	49.53	49.51	0.131	1.99	0.99	24.8
4	29.09	29.06	0.623	3.39	0.85	42.3
8	19.97	19.93	0.749	4.93	0.62	61.7
12	18.60	18.54	0.972	**5.29**	0.44	66.3
16	18.90	18.80	1.195	5.21	0.33	65.4
20	19.50	19.39	1.574	5.05	0.25	63.4
24	19.26	19.11	2.705	5.11	0.21	64.3

T_{comm} incluye la espera en las colectivas por el proceso más lento (desbalance). El `Bcast` puro de X_{tr} (83 MB) cuesta 0.15 s con $s = 24$: 0.8 % del total.